

* Regular Expressions in Python *

Index

- => Purpose of Regular Expression:-
- => Definition of Regular Expression.
- => Application of Regular Expression.
- => Module name required for Regular Expression -- re
- => The functions in "re" module
 - a) search()
 - b) findall()
 - c) finditer()
 - d) split()
 - e) end()
 - f) group
- => Programming examples.
- => Programmer Defined Character Classes.
- => Programming Examples
- => Pre-defined Character Classes.
- => Programming examples.
- => Quantifiers.
- => Examples.
- => Combined programming examples on programmer, pre-defined & Quantifiers

* Regular Expressions in Python *

- The Regular Expressions is an independent concept from programming language.
- The Regular Expressions concepts implemented by various programming language.
- In Python programming, Regular Expressions concepts implemented in the form of a module called "re"
- The purpose of Regular Expressions in Python is that "to validate the data and to build Robust Applications".

* Definition:

- A Regular expression is one of the search pattern which is a combination alphabets, digits and symbols which is used to find or search or match in given data obtaining desired result.

* Applications of Regular Expressions:-

- Regular Expressions are used in development of language compilers and interpreters.
- Regular Expressions are used in development of OSes.
- Regular Expressions are used in development of universal protocol like http, https, smtp, pop... etc.
- Regular Expressions are used in development of Electronic Circuits.
- Regular Expressions are used in development of pattern matching ... etc.
- To deal with Regular Expressions, we must import a pre-defined module called re.

* The pre-defined Functions in re module

- The 're' module contains the following essential functions.

1) finditer():-

syntax) Varname = re.finditer("search-pattern", "Given data")

- ⇒ Here, Varname is an object of type <class, 'callable_iterator'>
- ⇒ This function is used for searching the "search pattern" in given data iteratively and it returns table of entries which contains start index, end index and matched value based on the search pattern.

2) findall():-

syntax) Varname = re.findall("search-pattern", "Given data")

- ⇒ Here, Varname is an object of type <class, 'list'>
- ⇒ This function is used for searching the search pattern in entire given data and find all occurrences/matches and it returns all the matched values in the form an object <class, 'list'> but not returning start and End Index values.

3) search():

syntax) Varname = re.search("search-pattern", "Given data"),

⇒ Here Varname is an object of <class, 're.Match'> or <class, 'NoneType'>

⇒ this function is used for searching the search pattern in given data for first occurrence/match only but not for other occurrences/matches.

⊕ ⇒ if the search pattern found in given data then it returns an object of match class which contains matched value and start and end index values and it indicates search is successful.

⇒ if the search pattern is not found in given data then it returns None which is type <class, 'NoneType'> and it indicates search is un-successful.

4) group():

→ this function is used obtaining matched value by the finditer() and search()

→ this function present in match class of re-module.

syntax) Varname = matchobj.group(),

5) start():

→ this function is used obtaining starting index of matched value.

→ this function present in match class of re-module.

syntax) Varname = matchobj.start(),

6) end():-

→ this function is used for obtaining end index+1 of matched value

→ this function present in match class of re module

syntax) Varname = matchobj.end(),

7) sub() function:

→ This function replaces the matches with the text of your choice.

```
=> import re
txt = "the rain in spain"
x = re.sub("\s", "q", txt)
print(x)
```

RegExPl)

ex2)

```
import re
gd = "python is an oop lang, python is also fun lang"
```

ex3)

```
sp = "python"
```

```
noc = re.findall(sp, gd)
```

```
print("'{s}' found {t} times".format(*args: sp, len(noc)))
```

ex4)

ex4)

```
matd = re.finditer(sp, gd)
```

```
for mat in matd:
```

```
print("start index: {s} End index: {t} value: {v}"
```

```
format(mat.start(), mat.end(),
mat.group()))
```

ex1)

```
import re
```

```
gd = "python is an oop lang, python is also fun prog lang"
```

```
sp = "python"
```

```
res = re.search(sp, gd)
```

```
print(res)
```

ex2)

same

```
if (res != None):
```

```
print("search is successful")
```

```
print("start index: {s} value: {v}"
```

```
format(res.start(), res.end(),
res.group()))
```

```
else:
```

```
print("search is un-successful")
```

* Programmer-Defined Character Classes:-

→ The programmer-Defined Character Classes are defined or Designed by programmers for Defining OR Designing Search patterns which is used to search OR match or find in given data and obtains desired Result.

→ In other words programmer-Defined Character Classes are used in defining search patterns.

syntax) "[programmer-Defined Character Classes]"

→ The complete programmer-Defined Character Classes are given below.

1. [abc] → Searches for either 'a' or 'b' or 'c' only
2. [^abc] → Searches for all except 'a' or 'b' or 'c'
3. [A-Z] → searches for all upperCase Alphabets only
4. [^A-Z] → Searches for all except upper case Alphabets.
5. [a-z] → Searches for all lowerCase Alphabets only.
6. [^a-z] → Searches for all except lowerCase Alphabets.
7. [0-9] → searches for all digits only.
8. [^0-9] → Searches for all except Digits.
9. [A-Za-z] → Searches for both upper & lowerCase Alphabets.
10. [^A-Za-z] → Searches for All except upper & lowerCase Alphabets.
11. [A-Za-z0-9] → Searches for All Alphanumeric Values (other than special symbols)
12. [^A-Za-z0-9] → Searches for special symbols.
13. [A-Z0-9] → searches for upperCase Alphabets and Digits.
14. [^A-Z0-9] → searches for all except upperCase Alphabets and Digits
15. [a-z0-9] → Searches for lowerCase Alphabets and Digits.
16. [^a-z0-9] → searches for all except lowerCase Alphabets and Digits.

```

ex5) import re
mattab = re.finditer("[abc]", "cAk@2aLp4#G1q8HbQw")
print(" - - - ")
for mat in mattab:
    print("start index: {} End index: {} value: {}".format(mat.start(), mat.end(), mat.group()))
print(" - - - ")

```

output) start index : 0 End index: 1 Value c
 5 6 a
 14 15 b.

```

ex6) mattab = re.finditer("[^abc]", "cAk@2aLp4#G1q8HbQw")
ex7) mattab = re.finditer("[A-Z]", "cAk@2aLp4#G1q8HbQw")
ex8) mattab = re.finditer("[^A-Z]", "cAk@2aLp4#G1q8HbQw")
ex9) mattab = re.finditer("[a-z]", "cAk@2aLp4#G1q8HbQw")
ex10) mattab = re.finditer("[^a-z]", "cAk@2aLp4#G1q8HbQw")
ex11) mattab = re.finditer("[0-9]", "cAk@2aLp4#G1q8HbQw")
ex12) mattab = re.finditer("[^0-9]", "cAk@2aLp4#G1q8HbQw")
ex13) mattab = re.finditer("[A-Za-z0-9]", "cAk@2aLp4#G1q8HbQw")
ex14) mattab = re.finditer("[^A-Za-z0-9]", "cAk@2aLp4#G1q8HbQw")
ex15) mattab = re.finditer("[A-Za-z]", "cAk@2aLp4#G1q8HbQw")
ex16) mattab = re.finditer("[^A-Za-z]", "cAk@2aLp4#G1q8HbQw")
ex17) mattab = re.finditer("[A-z0-9]", "cAk@2aLp4#G1q8HbQw")
ex18) mattab = re.finditer("[^A-z0-9]", "cAk@2aLp4#G1q8HbQw")
ex19) mattab = re.finditer("[a-z0-9]", "cAk@2aLp4#G1q8HbQw")
ex20) mattab = re.finditer("[^a-z0-9]", "cAk@2aLp4#G1q8HbQw")

```

```

In [1]: #RegExpr1
import re
gd="Python is an oop lang.Python is also fun lang"
sp="Python"
noc=re.findall(sp,gd)
print("{} found {} Times".format(sp,len(noc)))

```

'Python' found 2 Times

```
In [2]: #RegExpr2
import re
gd="Python is an oop lang.Python is also fun lang"
sp="Python"
matd=re.finditer(sp,gd) # here matd is an object of <class, callable_iterator>
for mat in matd: # here mat is an object of <class, re.match>
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
```

Start Index:0 End Index:6 Value:Python
Start Index:22 End Index:28 Value:Python

```
In [3]: #RegExpr3
import re
gd="Python is an oop lang.python is also fun lang"
sp="Python"
res=re.search(sp,gd) # here res is an object of <class, re.match> if data found
if(res!=None):
    print("Search is Successful")
    print("Start Index:",res.start())
    print("End Index:",res.end())
    print("Value:",res.group())
else:
    print("Search is Un-Successful")
```

Search is Successful
Start Index: 0
End Index: 6
Value: Python

```
In [4]: #Searches for either 'a' or 'b' or 'c' only
#RegExpr4
import re
gd="cH4LuaP7#Bb6@QpWmD9%"
sp="[abc]"
matd=re.finditer(sp,gd)
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")
```

Start Index:0 End Index:1 Value:c
Start Index:5 End Index:6 Value:a
Start Index:10 End Index:11 Value:b

```
In [5]: #Searches for all except 'a' or 'b' or 'c' only
#RegExpr5
import re
matd=re.finditer("[^abc]","cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")
```

```
-----
Start Index:1 End Index:2 Value:H
Start Index:2 End Index:3 Value:4
Start Index:3 End Index:4 Value:L
Start Index:4 End Index:5 Value:u
Start Index:6 End Index:7 Value:P
Start Index:7 End Index:8 Value:7
Start Index:8 End Index:9 Value:#
Start Index:9 End Index:10 Value:B
Start Index:11 End Index:12 Value:6
Start Index:12 End Index:13 Value:@
Start Index:13 End Index:14 Value:Q
Start Index:14 End Index:15 Value:p
Start Index:15 End Index:16 Value:W
Start Index:16 End Index:17 Value:m
Start Index:17 End Index:18 Value:D
Start Index:18 End Index:19 Value:9
Start Index:19 End Index:20 Value:%
-----
```

```
In [6]: #Searches for all Upper Case alphabets
#RegExpr6
import re
matd=re.finditer("[A-Z]","cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")
```

```
-----
Start Index:1 End Index:2 Value:H
Start Index:3 End Index:4 Value:L
Start Index:6 End Index:7 Value:P
Start Index:9 End Index:10 Value:B
Start Index:13 End Index:14 Value:Q
Start Index:15 End Index:16 Value:W
Start Index:17 End Index:18 Value:D
-----
```

```
In [7]: #Searches for all except Upper Case alphabets
#RegExpr7
import re
matd=re.finditer("[^A-Z]","cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")
```



```
-----
Start Index:0 End Index:1 Value:c
Start Index:2 End Index:3 Value:4
Start Index:4 End Index:5 Value:u
Start Index:5 End Index:6 Value:a
Start Index:7 End Index:8 Value:7
Start Index:8 End Index:9 Value:#
Start Index:10 End Index:11 Value:b
Start Index:11 End Index:12 Value:6
Start Index:12 End Index:13 Value:@
Start Index:14 End Index:15 Value:p
Start Index:16 End Index:17 Value:m
Start Index:18 End Index:19 Value:9
Start Index:19 End Index:20 Value:%
-----
```

```
In [8]: #Searches for all Lower Case alphabets
#RegExpr8
import re
matd=re.finditer("[a-z]","cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:c
Start Index:4 End Index:5 Value:u
Start Index:5 End Index:6 Value:a
Start Index:10 End Index:11 Value:b
Start Index:14 End Index:15 Value:p
Start Index:16 End Index:17 Value:m
-----
```

```
In [9]: #Searches for all except Lower Case alphabets
#RegExpr9
import re
matd=re.finditer("[^a-z]","cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:1 End Index:2 Value:H
Start Index:2 End Index:3 Value:4
Start Index:3 End Index:4 Value:L
Start Index:6 End Index:7 Value:P
Start Index:7 End Index:8 Value:7
Start Index:8 End Index:9 Value:#
Start Index:9 End Index:10 Value:B
Start Index:11 End Index:12 Value:6
Start Index:12 End Index:13 Value:@
Start Index:13 End Index:14 Value:Q
Start Index:15 End Index:16 Value:W
Start Index:17 End Index:18 Value:D
Start Index:18 End Index:19 Value:9
Start Index:19 End Index:20 Value:%
-----
```

```
In [10]: #Searches for all digits
#RegExpr10
import re
matd=re.finditer("[0-9]","cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:2 End Index:3 Value:4
Start Index:7 End Index:8 Value:7
Start Index:11 End Index:12 Value:6
Start Index:18 End Index:19 Value:9
-----
```

```
In [11]: #Searches for all except digits
#RegExpr11
import re
matd=re.finditer("[^0-9]","cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:c
Start Index:1 End Index:2 Value:H
Start Index:3 End Index:4 Value:L
Start Index:4 End Index:5 Value:u
Start Index:5 End Index:6 Value:a
Start Index:6 End Index:7 Value:P
Start Index:8 End Index:9 Value:#
Start Index:9 End Index:10 Value:B
Start Index:10 End Index:11 Value:b
Start Index:12 End Index:13 Value:@
Start Index:13 End Index:14 Value:Q
Start Index:14 End Index:15 Value:p
Start Index:15 End Index:16 Value:W
Start Index:16 End Index:17 Value:m
Start Index:17 End Index:18 Value:D
Start Index:19 End Index:20 Value:%
-----
```

```
In [12]: #Searches for all only Alphabtes
#RegExpr12
import re
matd=re.finditer("[A-Za-z]","cH4LuaP7#Bb6@QpWmD9%")
noa=0
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
    noa=noa+1
print("-----")
print("Number of alphabets=",noa)
```

```
-----
Start Index:0 End Index:1 Value:c
Start Index:1 End Index:2 Value:H
Start Index:3 End Index:4 Value:L
Start Index:4 End Index:5 Value:u
Start Index:5 End Index:6 Value:a
Start Index:6 End Index:7 Value:P
Start Index:9 End Index:10 Value:B
Start Index:10 End Index:11 Value:b
Start Index:13 End Index:14 Value:Q
Start Index:14 End Index:15 Value:p
Start Index:15 End Index:16 Value:W
Start Index:16 End Index:17 Value:m
Start Index:17 End Index:18 Value:D
-----
```

Number of alphabets= 13

```
In [13]: #Searches for all except Alphabtes
#RegExpr13
import re
matd=re.finditer("[^A-Za-z]", "cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")
```

```
-----
Start Index:2 End Index:3 Value:4
Start Index:7 End Index:8 Value:7
Start Index:8 End Index:9 Value:#
Start Index:11 End Index:12 Value:6
Start Index:12 End Index:13 Value:@
Start Index:18 End Index:19 Value:9
Start Index:19 End Index:20 Value:%
-----
```

```
In [14]: #Searches for all alpha-numeric
#RegExpr14
import re
matd=re.finditer("[A-Za-z0-9]", "cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:c
Start Index:1 End Index:2 Value:H
Start Index:2 End Index:3 Value:4
Start Index:3 End Index:4 Value:L
Start Index:4 End Index:5 Value:u
Start Index:5 End Index:6 Value:a
Start Index:6 End Index:7 Value:P
Start Index:7 End Index:8 Value:7
Start Index:9 End Index:10 Value:B
Start Index:10 End Index:11 Value:b
Start Index:11 End Index:12 Value:6
Start Index:13 End Index:14 Value:Q
Start Index:14 End Index:15 Value:p
Start Index:15 End Index:16 Value:W
Start Index:16 End Index:17 Value:m
Start Index:17 End Index:18 Value:D
Start Index:18 End Index:19 Value:9
-----
```

```
In [15]: #Searches for all special symbols
#RegExpr15
import re
matd=re.finditer("[^A-Za-z0-9]", "!cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:!
Start Index:9 End Index:10 Value:#
Start Index:13 End Index:14 Value:@
Start Index:20 End Index:21 Value:%
-----
```

```
In [16]: #RegExpr16
import re
matd=re.finditer("[A-z0-9]", "!cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```



```
-----
Start Index:1 End Index:2 Value:c
Start Index:2 End Index:3 Value:H
Start Index:3 End Index:4 Value:4
Start Index:4 End Index:5 Value:L
Start Index:5 End Index:6 Value:u
Start Index:6 End Index:7 Value:a
Start Index:7 End Index:8 Value:P
Start Index:8 End Index:9 Value:7
Start Index:10 End Index:11 Value:B
Start Index:11 End Index:12 Value:b
Start Index:12 End Index:13 Value:6
Start Index:14 End Index:15 Value:Q
Start Index:15 End Index:16 Value:p
Start Index:16 End Index:17 Value:W
Start Index:17 End Index:18 Value:m
Start Index:18 End Index:19 Value:D
Start Index:19 End Index:20 Value:9
-----
```

```
In [17]: #RegExpr17
import re
matd=re.finditer("[^A-z0-9]", "!cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:!
Start Index:9 End Index:10 Value:#
Start Index:13 End Index:14 Value:@
Start Index:20 End Index:21 Value:%
-----
```

```
In [18]: #RegExpr18
import re
matd=re.finditer("[a-z0-9]", "!cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:1 End Index:2 Value:c
Start Index:3 End Index:4 Value:4
Start Index:5 End Index:6 Value:u
Start Index:6 End Index:7 Value:a
Start Index:8 End Index:9 Value:7
Start Index:11 End Index:12 Value:b
Start Index:12 End Index:13 Value:6
Start Index:15 End Index:16 Value:p
Start Index:17 End Index:18 Value:m
Start Index:19 End Index:20 Value:9
-----
```

```
In [20]: #RegExpr19
import re
matd=re.finditer("[^a-z0-9]", "!cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
```

```
print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en  
print("-----"))
```

```
-----  
Start Index:0 End Index:1 Value:!  
Start Index:2 End Index:3 Value:H  
Start Index:4 End Index:5 Value:L  
Start Index:7 End Index:8 Value:P  
Start Index:9 End Index:10 Value:#  
Start Index:10 End Index:11 Value:B  
Start Index:13 End Index:14 Value:@  
Start Index:14 End Index:15 Value:Q  
Start Index:16 End Index:17 Value:W  
Start Index:18 End Index:19 Value:D  
Start Index:20 End Index:21 Value:%  
-----
```

In []: